

Project Title: Smart Factory Equipment Anomaly Alert Agent

Position: ML Engineer, Pegatron

Assignment: Assignment 3

Vaibhav Kumar Sunkaria (<https://github.com/vaibhavsunkaria97/smart-factory-agent>)

DATE: 16/08/2026

1. Abstract

The present work is regarding processing an AI agent that monitors smart-factory equipment sensor data, detects anomalies automatically and delivers actionable real-time alerts. The pipeline generates a dummy sensor dataset (temperature, pressure, vibration), cleans it, scores every reading with two independent detectors, explains each anomaly through an LLM, and emits a prioritized alert table plus a machine-readable JSON log for downstream MES integration.

The design constraint follows directly from the assignment brief: because it rewards a low usage barrier, the system runs with one command and no API key: `python run_all.py` regenerates the data, alerts, metrics, figures and test results. LLM reasoning is used when available and degrades gracefully to a deterministic expert reasoner when it is not.

2. Development Workflow (AI-driven)

The project has been developed with an agentic coding loop: Aider, an agentic coding tool, was used to drive a free-tier frontier model, reading the repository, writing files and committing to git. Manual effort is limited to specifying constraints, verifying every output by execution, debug root cause issues and taking over where the agent stalls. Full prompts and the defect log are in `docs/AI_LOG.md` and `docs/aider_transcript.md` for free AI tool usage evidence.

3. Methodology and Model Details

3.1 Data Generation and Preprocessing

The equipment/machine generates sensor data (temperature, pressure, vibration) of 300 rows at one-minute intervals, out of which ~11% are abnormal with 1–3 channels corrupted per abnormal row. Missing values and duplicate timestamps have been injected deliberately, so that the cleaning step has real work to do. A generator-side assertion guarantees every row labelled abnormal, breaches at least one threshold. Preprocessing de-duplicates, imputes with time-aware interpolation, and produces a separate z-scored copy for the ML detector. Physical units are kept apart from scaled values, so alerts stay human-readable.

3.2 Detection Ensemble

A rule detector applies the brief's physical thresholds (temp $>52/<43$ °C, pressure $>1.08/<0.97$ bar, vibration >0.07 g), which will make it fast, explainable and it does not depend on past data. An Isolation Forest (200 trees) learns the normal multivariate envelope unsupervised and catches patterns that no single threshold expresses. Scores are fused as an OR-of-evidence (score = max (rule, ML)), favoring recall because a missed fault costs more than a false alarm. Severity follows from breach count and score.

3.3 Reasoning Layer

Each anomaly is turned into a diagnosis, and action is recommended by the best available free backend — local Ollama, free-tier Groq, or a deterministic knowledge base keyed on (signal, direction). The LLM never alters a detection; it only phrases the explanation, so correctness remains deterministic and unit tested.

4. System Architecture

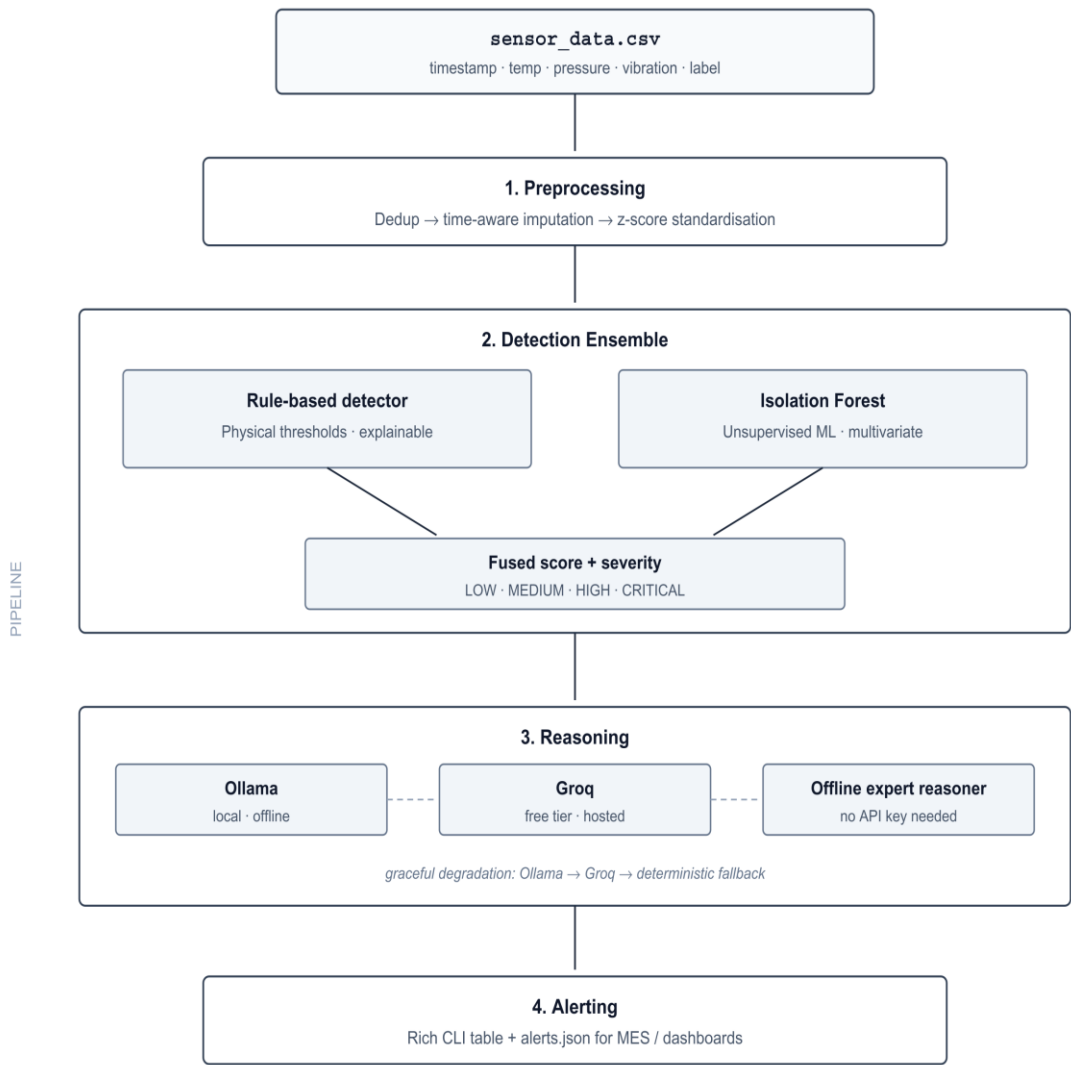


Fig. 1 — Smart Factory Agent processing pipeline

Figure 1: Four-stage pipeline with graceful degradation at the reasoning layer.

5. Evaluation

Four different detection algorithms were compared against ground-truth labels (300 readings, 33 true anomalies) and the results are shown in Table 1. F2 is reported alongside F1 because $\beta=2$ weights recall four times more than precision, which is the right emphasis for equipment safety. Accuracy is deliberately omitted: with ~11% positives, predicting 'all normal' would already score ~89%.

Table 1: Detector comparison. All configurations achieve perfect recall.

Detector	Precision	Recall	F1	F2	Flagged
Rule-based only	0.943	1.000	0.971	0.988	35
IsolationForest only	0.917	1.000	0.957	0.982	36
Gaussian Mixture (K=2)	0.917	1.000	0.957	0.982	36
Rule + IF ensemble (shipped)	0.917	1.000	0.957	0.982	36

6. Findings and Conclusion

Preprocessing manufactured two anomalies. Precision is below 1.0 for every detector, and investigations showed why. Two rows with a missing vibration reading sat between high-vibration neighbors; time-interpolation filled them with 0.080 and 0.088, both above the 0.07 limit. Those rows are labelled normal but breach after cleaning, so they count as false positives. The detectors behaved correctly — the imputed value never occurred. Cleaning is not a neutral step. In production the fix is causal forward-fill (which the streaming path requires anyway) plus flagging imputed values in the alert so an operator knows the reading was inferred rather than measured (refer Figure 2).

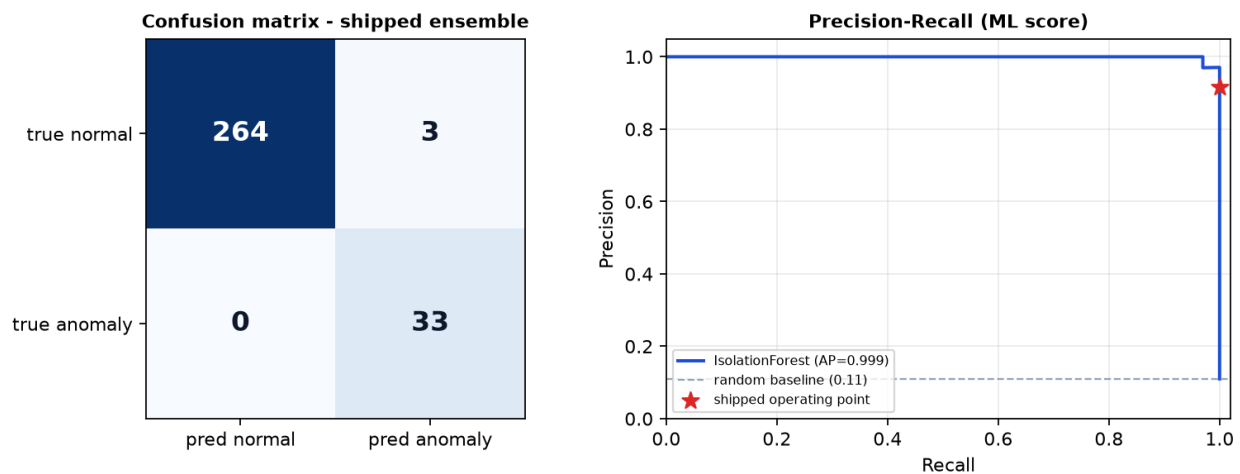


Figure 2: Confusion matrix and precision–recall curve with the shipped operating point.

7. Demonstration of Proposed Model Deliverable

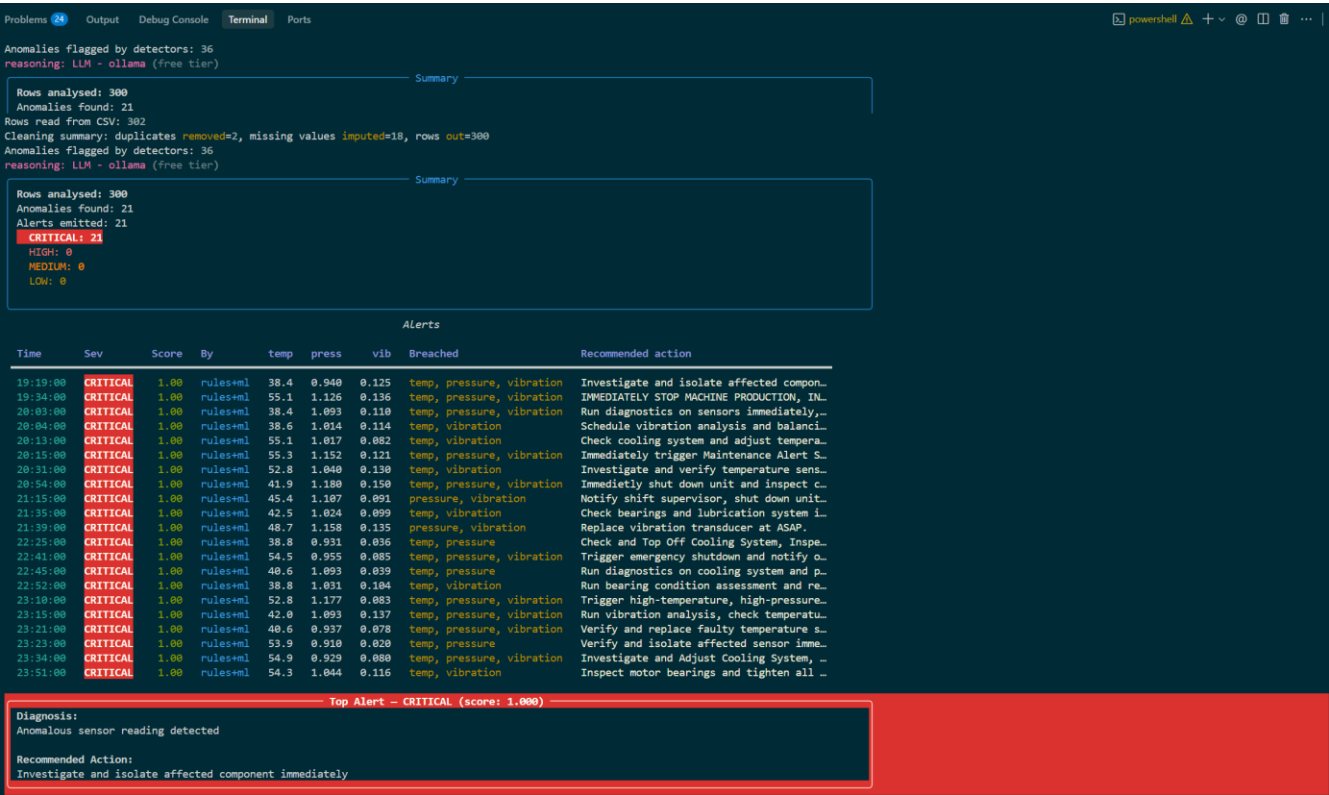


Figure 3: CRITICAL alerts with live Ollama reasoning: cleaning summary, prioritized table, and top-priority alert card.

```
(.venv) PS C:\Users\Vaibhav\smart-factory-agent> python agent.py --min-severity CRITICAL --backend offline
Anomaly Alert Agent

Rows read from CSV: 382
Cleaning summary: duplicates removed=2, missing values imputed=18, rows out=300
Anomalies flagged by detectors: 36
reasoning: deterministic expert reasoner (start Ollama or set GROQ_API_KEY for LLM reasoning)

Summary
Rows analysed: 300
Anomalies found: 21
Alerts emitted: 21
CRITICAL: 21
HIGH: 0
MEDIUM: 0
LOW: 0

Alerts
Time    Sev    Score  By      temp  press  vib    Breached    Recommended action
19:19:00 CRITICAL 1.00 rulesml 38.4  0.948  0.125  temp, pressure, vibration  Confirm the line has reached steady sta.
19:34:00 CRITICAL 1.00 rulesml 55.1  1.126  0.136  temp, pressure, vibration  Verify coolant pump flow and filter; in_
20:03:00 CRITICAL 1.00 rulesml 38.4  1.093  0.110  temp, pressure, vibration  Confirm the line has reached steady sta.
20:04:00 CRITICAL 1.00 rulesml 38.6  1.014  0.114  temp, vibration            Confirm the line has reached steady sta.
20:13:00 CRITICAL 1.00 rulesml 55.1  1.017  0.082  temp, vibration            Verify coolant pump flow and filter; in_
20:15:00 CRITICAL 1.00 rulesml 55.3  1.152  0.121  temp, pressure, vibration  Verify coolant pump flow and filter; in_
20:31:00 CRITICAL 1.00 rulesml 52.8  1.048  0.130  temp, vibration            Verify coolant pump flow and filter; in_
20:54:00 CRITICAL 1.00 rulesml 41.9  1.180  0.150  temp, pressure, vibration  Confirm the line has reached steady sta.
21:15:00 CRITICAL 1.00 rulesml 45.4  1.107  0.091  pressure, vibration        Inspect relief valve and outlet line; v_
21:35:00 CRITICAL 1.00 rulesml 42.5  1.024  0.099  temp, vibration            Confirm the line has reached steady sta.
21:39:00 CRITICAL 1.00 rulesml 48.7  1.158  0.135  pressure, vibration        Inspect relief valve and outlet line; v_
22:25:00 CRITICAL 1.00 rulesml 38.8  0.931  0.036  temp, pressure            Confirm the line has reached steady sta.
22:41:00 CRITICAL 1.00 rulesml 54.5  0.955  0.085  temp, pressure, vibration  Verify coolant pump flow and filter; in_
22:45:00 CRITICAL 1.00 rulesml 40.6  1.093  0.039  temp, pressure            Confirm the line has reached steady sta.
22:52:00 CRITICAL 1.00 rulesml 38.8  1.031  0.104  temp, vibration            Confirm the line has reached steady sta.
23:10:00 CRITICAL 1.00 rulesml 52.8  1.177  0.083  temp, pressure, vibration  Verify coolant pump flow and filter; in_
23:15:00 CRITICAL 1.00 rulesml 42.0  1.093  0.137  temp, pressure, vibration  Confirm the line has reached steady sta.
23:21:00 CRITICAL 1.00 rulesml 40.6  0.937  0.078  temp, pressure, vibration  Confirm the line has reached steady sta.
23:23:00 CRITICAL 1.00 rulesml 53.9  0.918  0.020  temp, pressure            Verify coolant pump flow and filter; in_
23:34:00 CRITICAL 1.00 rulesml 54.9  0.929  0.080  temp, pressure, vibration  Verify coolant pump flow and filter; in_
23:51:00 CRITICAL 1.00 rulesml 54.3  1.044  0.116  temp, vibration            Verify coolant pump flow and filter; in_

Top Alert - CRITICAL (score: 1.000)
Diagnosis:
Under-temperature: sensor fault, incomplete warm-up, or chiller over-cooling. Under-pressure: leak, pump cavitation, or supply loss. Excess vibration: bearing wear, shaft misalignment, or loosened mounting.
Recommended Action:
Confirm the line has reached steady state; validate the sensor against a reference probe. Leak-check seals and fittings; confirm supply pressure; inspect pump for cavitation. Schedule bearing inspection; check shaft alignment and mounting bolt torque.

Alerts saved to alerts.json
(.venv) PS C:\Users\Vaibhav\smart-factory-agent>
```

Figure 4: Offline fallback execution with deterministic expert reasoner output when no LLM backend is active.

8. Use of AI Tools

Only free tier AI was used for project constraints. AI appears in two distinct roles: tools that built the system, and an LLM embedded inside the delivered product as its reasoning layer. (refer Table 2)

Table 2. AI Model Contribution

Free AI tool	Type	Contribution
Aider + free-tier frontier model	Build-time	Agentic loop: read repo, planned changes, wrote modules and tests, commits.
Claude (free tier)	Build-time	Requirement analysis, architecture decisions, debugging, report generation.
Ollama / llama3.2 (local)	Runtime	LLM inside the product writing each alert's diagnosis — free, offline, air-gap friendly.
Groq (free tier)	Runtime	Hosted alternative reasoning backend.

8.1 AI as Reviewer, Not Only Generator

Every AI-generated module was executed and its output inspected rather than reviewed by reading alone. This process identified five defects, none of which were apparent from the source code:

1. **Invalid ground-truth labels.** The data generator produced 8 of 41 rows labelled *abnormal* that breached no threshold which left uncorrected, these would have been permanent false negatives, capping recall at approximately 80%. The root cause was Gaussian noise applied after channel corruption, returning boundary values to the normal range. A generator-side assertion now enforces the invariant.
2. **Syntax error from a multi-part edit.** The model emitted a duplicated loop header and an incomplete assertion, producing a file that would not parse. Caught by validating syntax before execution.
3. **Type error in preprocessing.** Interpolation was applied to the full Data Frame including the string label column, raising a `TypeError` at runtime. The line is valid in isolation; failure depends on the column types present in the data.
4. **Scale-dependent injection failure. Missing-value injection produced zero NaN cells at 100 rows while producing 18 at 300, rendering the imputation step useless on small datasets. It was identified by a unit test executing the generator at a size that had never been run manually.**
5. **Silent failure at LLM backend.** The reasoning layer reported and returned **backend: ollama** output while every LLM call was failing. Requests were posted to the service root rather than the endpoint, and a bare `except: pass` suppressed the resulting exceptions which caused silent fallback to the deterministic reasoner only. The defect was identified from output quality, not from an error: two of three alerts returned generic text where specific guidance was expected. The last error motivated me to apply general principle throughout the system i.e., **status output must report observed state, not intended state**. The same class of error recurred in **agent.py** where the header displays the requested backend rather than the one selected.

8.2 Measured Limitations of the Local Model

The same 36 detections were processed through both llama3.2 and the deterministic knowledge base for comparison. Three failure modes were observed in the model output:

- **Incomplete diagnosis.** On rows breaching multiple thresholds, the model frequently addressed only one signal.
- **Directional inversion.** Recommendations occasionally contradicted the fault, including "increase operating temperature" for an under-temperature breach.
- **Hallucinated hardware.** Output referenced equipment is not present in the system, such as "Replace sensor 3".

The deterministic knowledge base exhibited none of these, as its recommendations are retrieved against detected faults rather than generated. This comparison is the basis for the architectural constraint that the LLM never influences detection and is confined to phrasing explanations, leaving classification deterministic and unit testable.

9. Limitations of Proposed Model and Next Steps

1. **Point anomalies** - Each reading is scored independently, so the system cannot detect faults that manifest as a pattern across time — gradual drift, or a subsequence that is abnormal despite every individual reading remaining within limits. Detecting these requires windowed or sequence models.
2. A sequence model was deliberately not attempted for the project. The synthetic dataset consists of independently sampled point anomalies with no temporal correlation, so an LSTM model would have no learnable temporal structure. Demonstrating collective-anomaly detection requires data containing collective anomalies.

3. **Non-causal imputation.** Missing values are filled by time-aware interpolation, which estimates a gap from the readings on both sides. This is valid for batch analysis of historical data but invalid in a streaming context, where the subsequent reading has not yet occurred.
4. **Heuristic severity thresholds.** The severity boundaries (0.85 for CRITICAL, 0.60 for HIGH) were selected to produce a usable distribution across the dataset. They are not derived from a cost model. A production implementation would map severity to expected operational impact, defined jointly with process owners.

Next Steps:

1. **Streaming ingestion.** Replace the CSV source file with a Kafka consumer, enabling continuous streaming and monitoring rather than post batch analysis. Detection latency is already sub-millisecond; LLM enrichment would run asynchronously off the critical path.
2. **Per-equipment thresholds.** Operating envelopes are currently global. Thresholds should be derived per machine from its own operating history, since baseline behavior varies with equipment age and configuration.
3. **Drift detection and retraining.** The definition of normal changes as equipment ages and process recipes are revised. Monitoring the score distribution and alert rate would trigger scheduled retraining before the model degrades silently.
4. **Operator feedback loop.** Capturing confirm/dismiss decisions on each alert accumulates genuine fault labels over time. This provides the labelled dataset required to train a supervised model on observed faults, rather than on labels derived from the thresholds already encoded in the rule layer.
5. **MES integration.** A CRITICAL alert should automatically raise a maintenance work order, closing the loop between detection and intervention.